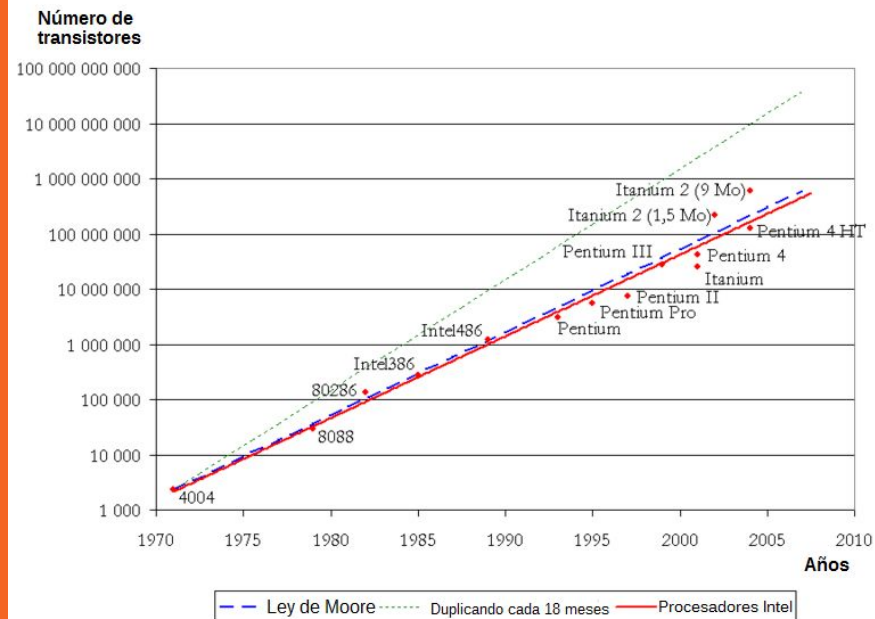

OpenMP: Open Multi-Processing

Contexto

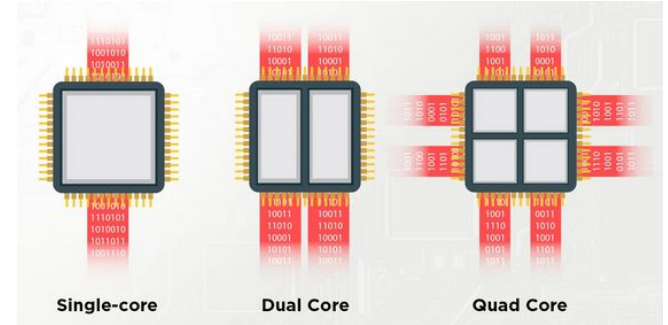
Ley de Moore



Contrato hardware/software



Hardware -> Nuevas arquitecturas



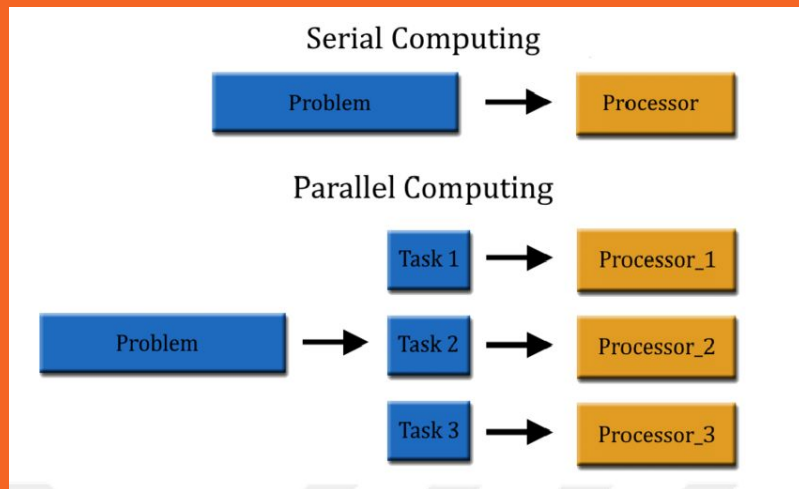
Software -> Mejorar performance -> OpenMP



OpenMP

Una API para programar aplicaciones multihilo

- Un conjunto de directivas de compilador y bibliotecas de rutinas para programación de aplicaciones paralelas
- Simplifica la escritura de programas multihilo en Fortran, C y C++.



Características



- OpenMP se caracteriza por ser fácil de aprender, flexible y portátil.
- El código fuente de una aplicación puede ser adaptado a diferentes sistemas sin tener que modificar el código fuente original.
- Ofrece una gran variedad de directivas para la creación de hilos, la sincronización y la asignación de trabajo.

Características



- OpenMP se caracteriza por ser fácil de aprender, flexible y portátil.
- El código fuente de una aplicación puede ser adaptado a diferentes sistemas sin tener que modificar el código fuente original.
- Ofrece una gran variedad de directivas para la creación de hilos, la sincronización y la asignación de trabajo.

Características

```
void Suma(double* a, double* b, double* c, int size)
{
    for (int i = 0; i < size; ++i)
    {
        c[i] = a[i] + b[i];
    }
}
```

```
void Suma(double* a, double* b, double* c, int size)
{
    #pragma omp parallel for
    for (int i = 0; i < size; ++i)
    {
        c[i] = a[i] + b[i];
    }
}
```

Modelo de programación

```
main()
{
    /*.....*/

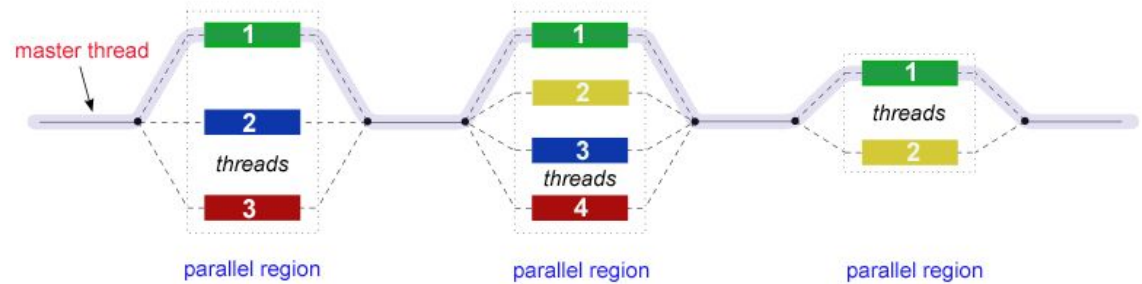
    #pragma omp parallel
    {
        /*.....
        .....
        .....*/
    }

    /*.....*/

    #pragma omp parallel
    {
        /*.....
        .....
        .....*/
    }

    /*.....*/

    #pragma omp parallel
    {
        /*.....
        .....
        .....*/
    }
}
```



Sintaxis

Directivas

```
#pragma omp parallel  
num_threads(4)
```

Funciones de biblioteca

```
omp_get_thread_num();  
  
omp_set_num_threads(2);  
  
double omp_get_wtime  
(void);
```

Variables de Entorno

```
export  
OMP_NUM_THREADS=4  
  
./programa
```

Directivas

- Son palabras clave que se utilizan en el código fuente para indicar a la aplicación qué secciones del código se deben ejecutar en paralelo.
- También se utilizan para especificar la creación y sincronización de hilos.

C hello_world.c > ...

```
1  #include <omp.h>
2
3  #include <stdio.h>
4  #include <stdlib.h>
5
6  int main(int argc, char* argv[])
7  {
8      #pragma omp parallel
9      {
10         int ID=omp_get_thread_num();
11
12         printf("Hello (%d) ",ID);
13         printf(" world (%d) \n",ID);
14     }
15 }
```

```
$ gcc -fopenmp -o hello_world.exe hello_world.c && ./hello_world.exe
```

```
Hello (2) world (2)
Hello (0) world (0)
Hello (1) world (1)
Hello (3) world (3)
```

Directivas

- **parallel:** Esta directiva crea una región paralela donde el trabajo se divide entre múltiples hilos. Cada hilo ejecuta una copia de la región paralela y luego se unen al final.
- **for:** La directiva for se utiliza para paralelizar bucles. Divide el trabajo del bucle entre los hilos disponibles, donde cada hilo ejecuta una porción del bucle.
- **sections:** Esta directiva se utiliza para paralelizar secciones de código independientes. Cada sección se ejecuta en un hilo diferente.
- **single:** Con la directiva single, una sección de código se ejecuta solo en un hilo. Se puede utilizar para inicializaciones o para realizar operaciones que deben realizarse una sola vez.

Directivas: parallel for

```
#include <stdio.h>
#include <omp.h>

int main()
{
    int suma = 0;

    #pragma omp parallel for
    for (int i = 1; i <= 100; i++)
    {
        suma += i;
    }
    printf("Suma = %d\n", suma);

    return 0;
}
```

Ejecución 1:

Suma = 3664

Ejecución 2:

Suma = 3833

Ejecución 3:

Suma = 4586

Ejecución 4:

Suma = 1778

Directivas: reduction (op, list)

```
int main()
{
    int suma = 0;

    #pragma omp parallel for reduction(+:suma)
    for (int i = 1; i <= 100; i++)
    {
        suma += i;
    }
    printf("Suma = %d\n", suma);

    return 0;
}
```

Ejecución 1:

Suma = 5050

Ejecución 2:

Suma = 5050

Ejecución 3:

Suma = 5050

Ejecución 4:

Suma = 5050

Directivas: Sections

```
#pragma omp parallel
{
    #pragma omp single
    printf("Suma = %d\n", suma);

    #pragma omp sections
    {
        #pragma omp section
        printf("Section 1 - hilo %d\n", omp_get_thread_num());

        #pragma omp section
        printf("Section 2 - hilo %d\n", omp_get_thread_num());

        #pragma omp section
        printf("Section 3 - hilo %d\n", omp_get_thread_num());

        #pragma omp section
        printf("Section 4 - hilo %d\n", omp_get_thread_num());
    }
}

return 0;
```

Ejecución 1:
Suma = 4361
Section 2 - hilo 2
Section 3 - hilo 1
Section 4 - hilo 3
Section 1 - hilo 0

Ejecución 2:
Suma = 3902
Section 3 - hilo 0
Section 4 - hilo 2
Section 2 - hilo 3
Section 1 - hilo 1

Ejecución 3:
Suma = 4244
Section 1 - hilo 0
Section 4 - hilo 0
Section 2 - hilo 1
Section 3 - hilo 2

Ejecución 4:
Suma = 5050
Section 1 - hilo 3
Section 3 - hilo 3
Section 4 - hilo 3
Section 2 - hilo 2

Directiva: Schedule

```
#include <stdio.h>
#include <omp.h>
int main()
{
    int suma = 0;
    #pragma omp parallel for
        for (int i = 1; i <= 100; i++)
        {
            suma += i;
        }
    printf("Suma = %d\n", suma);
    return 0;
}
```

0	25	50	75	100
Hilo 1	Hilo 2	Hilo 3	Hilo 4	

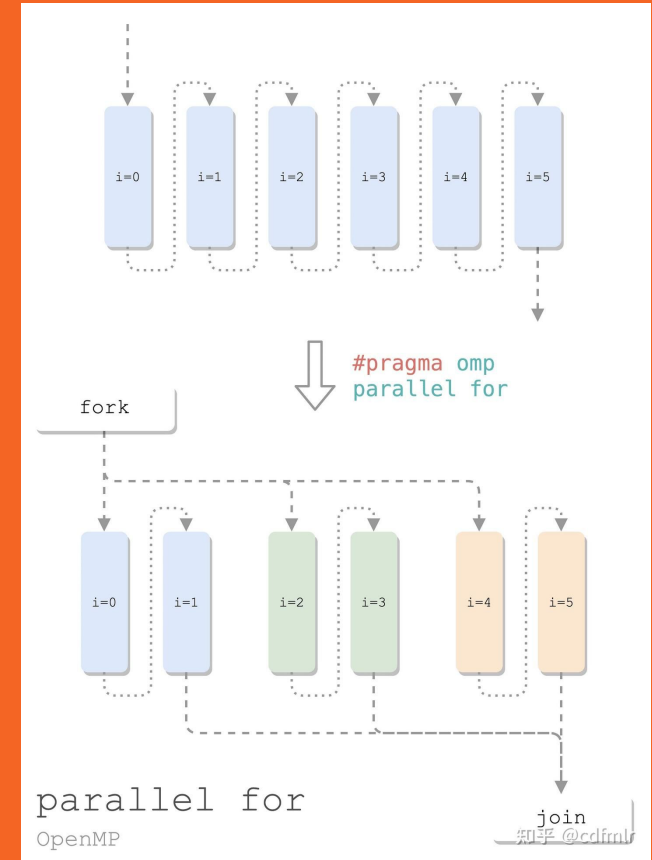
Thread 2: indice i = 55
Thread 1: indice i = 30
Thread 3: indice i = 80
Thread 0: indice i = 10
Suma = 4935

Thread 0: indice i = 10
Thread 3: indice i = 80
Thread 1: indice i = 30
Thread 2: indice i = 55
Suma = 5024

Thread 3: indice i = 80
Thread 1: indice i = 30
Thread 2: indice i = 55
Thread 0: indice i = 10
Suma = 4817

Directiva: Schedule

- **static** Las iteraciones se dividen en chunks de un tamaño definido.
- **dynamic** Cada thread ejecuta un chunk de iteraciones y al terminar su parte busca otro chunk para procesar.
- **guided** Cada thread ejecuta un chunk de iteraciones y al terminar su parte busca otro chunk para procesar. El tamaño del chunk varía, siendo grande al iniciar y éste se va reduciendo.
- **auto** La decisión del tipo de schedule es decidido por el compilador/sistema operativo.



Directivas

- **task:** La directiva task se utiliza para crear tareas independientes que pueden ser ejecutadas por hilos disponibles. Proporciona un modelo de ejecución más flexible que las directivas for y sections.
- **critical:** La directiva critical se utiliza para definir una sección crítica del código, donde solo un hilo puede ejecutarla a la vez. Se utiliza para proteger secciones de código que acceden a recursos compartidos.
- **atomic:** La directiva atomic se utiliza para realizar operaciones atómicas en variables compartidas. Garantiza que la operación se realice sin interferencias de otros hilos.
- **barrier:** La directiva barrier se utiliza para sincronizar todos los hilos en un punto específico del programa. Los hilos esperarán hasta que todos los demás hilos alcancen el mismo punto antes de continuar

Funciones de la biblioteca

OpenMP incluye una biblioteca de funciones que se utilizan para crear y administrar hilos, así como para controlar la sincronización y la exclusión mutua:

- **omp_get_thread_num():** Esta función devuelve el número de identificación del hilo que la llama. Los hilos se numeran de forma consecutiva, comenzando desde cero.
- **omp_get_num_threads():** Esta función devuelve el número total de hilos en la región paralela en la que se encuentra el hilo que la llama.
- **omp_set_num_threads():** Esta función se utiliza para establecer el número de hilos que se utilizarán en una región paralela. Se puede utilizar para ajustar la cantidad de paralelismo en función de los recursos disponibles.

Funciones de la biblioteca

```
#include <stdio.h>
#include <omp.h>

void ejecutar(int hilos) {
    omp_set_num_threads(hilos);

    #pragma omp parallel
    {
        printf("Hilo %d de %d\n",
               omp_get_thread_num(),
               omp_get_num_threads());
    }
    printf("\n");
}

int main() {
    ejecutar(2);
    ejecutar(4);
    ejecutar(6);
    return 0;
}
```

Hilo 0 de 2
Hilo 1 de 2

Hilo 2 de 4
Hilo 1 de 4
Hilo 3 de 4
Hilo 0 de 4

Hilo 1 de 6
Hilo 0 de 6
Hilo 5 de 6
Hilo 3 de 6
Hilo 2 de 6
Hilo 4 de 6

Funciones de la biblioteca

- **omp_get_num_procs():** Esta función devuelve el número de procesadores disponibles en el sistema.
- **omp_get_wtime():** Esta función devuelve el tiempo en segundos desde un punto de referencia específico. Se puede utilizar para medir el tiempo de ejecución de una región paralela o de una sección de código.
- **omp_get_max_threads():** Esta función devuelve el número máximo de hilos que se pueden utilizar en una región paralela. Puede ser útil para determinar el límite superior de paralelismo.
- **omp_get_dynamic():** Esta función devuelve el estado actual de la ajuste dinámico de hilos. Si está habilitado, el número de hilos se puede ajustar en tiempo de ejecución.
- **omp_set_dynamic():** Esta función se utiliza para habilitar o deshabilitar el ajuste dinámico de hilos. Cuando está habilitado, el número de hilos se puede ajustar en tiempo de ejecución.

Funciones de la biblioteca

```
#include <stdio.h>
#include <omp.h>

int main() {
    long n = 100000000;
    long suma = 0;
    double inicio, fin;

    printf("Maximos hilos disponibles: %d\n\n", omp_get_max_threads());

    // Suma secuencial
    inicio = omp_get_wtime();

    for (long i = 1; i <= n; i++)
        suma += i;

    fin = omp_get_wtime();

    printf("Secuencial: %ld\n", suma);
    printf("Tiempo: %f segundos\n\n", fin - inicio);

    // Suma paralela
    suma = 0;
    inicio = omp_get_wtime();

    #pragma omp parallel for reduction(+:suma)
    for (long i = 1; i <= n; i++)
        suma += i;

    fin = omp_get_wtime();

    printf("Paralela: %ld\n", suma);
    printf("Tiempo: %f segundos\n", fin - inicio);
}
```

Ejecución 1:
Maximos hilos disponibles: 4

Secuencial: 5050
Tiempo: 0.000000 segundos

Paralela: 5050
Tiempo: 0.002057 segundos

Ejecución 2:
Maximos hilos disponibles: 4

Secuencial: 5050
Tiempo: 0.000000 segundos

Paralela: 5050
Tiempo: 0.006956 segundos

Ejecución 3:
Maximos hilos disponibles: 4

Secuencial: 5050
Tiempo: 0.000001 segundos

Paralela: 5050
Tiempo: 0.001079 segundos

Ejecución 4:
Maximos hilos disponibles: 4

Secuencial: 5050
Tiempo: 0.000000 segundos

Paralela: 5050
Tiempo: 0.004136 segundos

Ejecución 1:
Maximos hilos disponibles: 4

Secuencial: 5000000050000000
Tiempo: 0.209473 segundos

Paralela: 5000000050000000
Tiempo: 0.070594 segundos

Ejecución 2:
Maximos hilos disponibles: 4

Secuencial: 5000000050000000
Tiempo: 0.203389 segundos

Paralela: 5000000050000000
Tiempo: 0.068422 segundos

Ejecución 3:
Maximos hilos disponibles: 4

Secuencial: 5000000050000000
Tiempo: 0.221109 segundos

Paralela: 5000000050000000
Tiempo: 0.062690 segundos

Ejecución 4:
Maximos hilos disponibles: 4

Secuencial: 5000000050000000
Tiempo: 0.223728 segundos

Paralela: 5000000050000000
Tiempo: 0.068627 segundos

Recursos

- <https://www.openmp.org/resources/openmp-presentations/>
 - <https://www.youtube.com/watch?v=nE-xN4Bf8XI&list=PL LX-Q6B8xqZ8n8bwjGdzBJ25X2utwnoEG&index=1>
 - https://github.com/tgmattso/OpenMP_intro_tutorial
 - https://www.openmp.org/wp-content/uploads/Intro_To_OpenMP_Mattson.pdf
-